PSAI_118

Epirus Substrate Explorer

- **Team Name:** Web3 Labs Ltd
- **Payment Address:** 0xc905c448db9942c662fcb1680f3ecfcd0592409c
- **Level:** 2

Project Overview :page_facing_up:

Overview

Epirus Substrate Explorer will be a modular explorer for Substrate-based blockchains centred on supporting the Substrate contracts pallet and ink! smart contracts.

Web3 Labs has developed a closed source commercial blockchain explorer for EVM blockchains named Epirus. It is already in production for some private chains of financial institutions, Chainstack, the Palm Network and Icon's upcoming ICE network. The acquired knowledge and the existing source code, if applicable, will speed up the delivery of the product.

Our aspirational vision is to deliver a fully modular system providing a runtime for end-to-end extension bundles to compose your bespoke blockchain explorer instance. Each bundle should include the data model, the API and the UI to serve a specific set of functionality available in a Substrate node (e.g. contracts, identity, treasury, etc.) and be installable from a repository. In addition to supporting pre-built pallets, it would serve as a powerful complement for custom pallets. This design could stimulate the evolution of the ecosystem through the open-source community.

Project Details

Architecture

The following diagram shows a high-level overview of the system logical components.



Epirus Substrate Architecture

The UI client application (C1) runs in a web browser and connects to a stateless web-accessible remote API through HTTP (C2).

The API exposes the database entities in an easy to query way using GraphQL (C2).

The database serves as a data enrichment layer and indexing of Substrate related data entities (i.e. metadata, blocks, extrinsics and events).

The blockchain data services is a modular event-driven data extraction and transformation pipeline. This design will allow us to scale out the processors and extension modules that compose a data pipeline.

The Substrate Indexer (C3) loads blocks from genesis to the present block height through a Substrate node RPC API. These blocks are stored in the database, emitted to an event hub, and consumed by the base processors (C4) responsible for extracting additional core entities.

There is a processor per core data type, e.g. events from chain storage, to extract, transform and load it into the database. Each base processor emits events for its extracted entities.

The extension modules are data processors with a function-specific processing scope, existing one per supported pallet. The Accounts module (C5) is an extension module to index data related to accounts and balances, while the Contracts module (C6) does the same for contract pallet related data.

System Components

ID	Name	Description
C1	Explorer UI	The Substrate Explorer client facing application that uses the Explorer API. The Explorer UI client application runs in the end-user web browser and uses the Explorer API (C2) for the rendition of user views. A registry of composable display blocks, for the navigation and the views itself will be implemented in order to make it easy to adapt the UI to the different pallets available in the Substrate node.
C2*	Explorer API	Stateless GraphQL API to access the underlying database. Supports searching by indexed properties, pagination and sorting of results. We plan to serve it using PostGraphile or Husura. The API services can scale out by the usual HTTP load balancing.
C3*	Substrate Indexer	Load blocks from a Substrate node. It is responsible for reaching the most recently finalised block in the chain and emitting events on the ingestion of blocks. This component is the entry point for the transformation pipeline (C4, C5 and C6).
C4*	Base Processors	Components for indexing and decoding the fundamental Substrate data types, i.e. blocks, extrinsics, events and logs.
C5	Accounts Module	Module to index accounts and balances information. This is a prerequisite for the Contracts module (C6).
C6	Contracts Module	Module to support the contracts pallet and ink! smart contracts. Features:

Notes * Depending on the readiness of the existing ecosystem projects for indexing and GraphQL API building, e.g. Substrate Archive, Subsquid or Subquery, could be used to provide the functionality of the API (C2), Substrate Indexer (C3) and Base Processors (C4).

In any case, we will align the project with existing ecosystem solutions as much as possible.

User Interface

The user interface will be based on the existing Epirus look & feel:



Epirus Look and Feel

Technology Stack

Web UI

- Typescript – Strongly typed programming language that builds on JavaScript.
- React / Preact – JavaScript library for building user interfaces. We will consider using Preact, the fast 3kB alternative to React with the same modern API, to reduce the package size if the bundler tree shaking is not enough.
- Tailwind CSS – A utility-first CSS framework packed with classes that can be composed to build any design, directly in your markup.

Data Processing***

- Kotlin – Modern, concise and safe programming language.
- Quarkus – Kubernetes native Java stack tailored for OpenJDK HotSpot and GraalVM, crafted from the best of breed Java libraries and standards.
- GraalVM – High-performance JDK distribution. It is designed to accelerate the execution of applications written in Java and other JVM languages while also providing runtimes for JavaScript, Ruby, Python, and a number of other popular languages. GraalVM's polyglot capabilities make it possible to mix multiple programming languages in a single application while eliminating any foreign language call costs.
- PolkaJ – Java client library to use and access API of Polkadot based networks.

***** *Depending on the maturity of the Substrate JVM ecosystem, we can switch to languages with more mature support for Substrate.*

Web API

- GraphQL – Query language for APIs and a runtime for fulfilling those queries with your existing data.
- PostGraphile – Builds a powerful, extensible and performant GraphQL API from a PostgreSQL schema.

Database

- PostgreSQL – Powerful open source object-relational database. Provides first-class support for JSON data types.
- Citus – PostgreSQL extension that transforms Postgres into a distributed database—so you can achieve high performance at any scale.

Synergies to Explore

- Substrate Archive – As a possibility to be evaluated, the Substrate Indexer and Base Processors could be provided by this Substrate blockchain indexing engine.
- Subsquid – A query node framework for Substrate-based blockchains. In very simple terms, Subsquid can be thought of as an ETL tool, with a GraphQL server included. It is alpha stage software.
- Subquery - A framework to define your own APIs to expose blockchain indexed data. It is similar to the Graph Network.

Ecosystem Fit

Substrate Epirus Explorer, in general, aims to deliver a great user experience while browsing blockchain information. In particular, it will provide extensive support for accessing all the information related to deployed WASM bytecode and its account instances.

The scope of this grant includes a baseline implementation of the core mechanisms and an elementary information display for contracts and accounts. The user interface will fulfil the minimum requirements for the prospective customers of the system to deliver a meaningful and fully functional baseline.

Audience

Name	Usages
Developers	
Service Providers	
End Users	

Existing Projects

There are other Substrate based blockchain explorers, notably Subscan and Polkascan. Both are general Web3 explorers, whilst Subscan provides a better user experience, supports some pre-build pallets (i.e. staking, EVM and parachain) and exposes a REST API.

We believe that a single provider cannot absorb all the UI experiences tailored to the growing number of pallets in the Substrate ecosystem. An open community package registry for this UI experience that exposes the functionality and data related to the pallets would benefit the evolution of the ecosystem. Regarding the API, the REST model imposes a rigid model to access information compared to GraphQL.

Epirus Substrate Explorer differentiates itself from the present offering in the following key points:

- Initial focus on smart contracts and ink! with unique and novel features like verification and standards contract identification.
- Ease of customisation and branding without tech provider lock-in.
- Aim for end-to-end modularity and extensibility.
- Aim to avoid duplication and foster synergies leveraging existing projects in the ecosystem, like Subquery or Subsquid.

Potential Synergies

These are some projects that could ease the implementation of the proposed system:

a) Substrate Archive is a project that decouples the indexing engine from the blockchain explorer, making it convenient to index historical blockchain data.

b) Subsquid is a project targeted at developers aiming to reduce the complexity of fetching and transforming blockchain data and exposing it through a GraphQL API.

c) Subquery is a project with similar functionality to Subsquid.

Subquery and Subsquid projects are potential candidates to serve as the indexing backend of Epirus Explorer. However, the indexed data needs to turn into the human-friendly information displayed in the explorer UI, requiring further transformations and data enrichment (e.g. metadata from ABI descriptors, RPC calls for standard contract interfaces, etc.) This data transformation and the user-friendly display are what Epirus Explorer provides.

Development Roadmap :nut_and_bolt:

Overview

- **Total Estimated Duration:** 2 months
- **Full-Time Equivalent (FTE):** 2 FTE
- **Total Costs:** 50,000 EUR

Milestone 1 - Devnet MVP

- **Estimated duration:** 2 months
- **FTE:** 2
- **Costs:** 50,000 EUR

Nº	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both inline documentation of the code and a basic tutorial that explains how a user can (for example) spin up an Explorer instance and connect it to a Substrate node.
0c.	Testing Guide	Core functions will be fully covered by unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0d.	Docker	We will provide a Dockerfile and Docker Compose file(s) to ease the deployment and execution of the system.
1.	Baseline implementation of the system components	The implementation of the system components described in the “System Components” section:
2.	Public explorer instance	A publically accessible instance of the Explorer connected to a development network with ink! smart contracts support.

For this minimum viable product delivery the following features are out of scope:

- Detection of standard contracts, e.g. Open Brush PSP22, PSP34, etc.
- Uploading verified ABI metadata
- Decoding of the contract calls based on ABI metadata
- Decoding of events based on ABI metadata
- Processing of “intrinsics”, i.e. cross-contract interactions equivalent to Ethereum “internal transactions”

Team :busts_in_silhouette:

Team Members

- Marc Fornós
- Xueying Wang

Contact

- **Contact Name:** Conor Svensson
- **Contact Email:** conor@web3labs.com

Legal Structure

- **Registered Address:** 7 Bell Yard, London, England, WC2A 2JR

- **Registered Legal Entity:** Web3 Labs Ltd, CRN 10783824

Team's Experience

Marc Fornós has been designing and implementing software systems for 20 years. He is an expert in the area of distributed systems and data-intensive applications. His experience ranges from warehouse automation with radio-frequency terminals using the Adaptive Communication Environment C++ framework, to airline post-sale revenue optimization software-as-a-service platform, generating millions in incremental revenue for low-cost and full-service carriers during eight years of operation. Currently in charge of evolving the Web3 Labs Epirus Block Explorer product offering.

Xueying Wang pivoted to software development after completing an MSc. in Aerospace Engineering and has been in the industry for the past six years. During this time, she pioneered conversational AI assistants for airlines, counting more than 20 assistants in production covering ten languages for customer service, FAQ and in-chat purchases. Also built a scalable publish-subscribe system to trigger actions on flight feed events for the automated agents. She participated in implementing a crypto trading bot as a side project and a composable Solid POD/RDF data browser. She has a great deal of experience developing React applications and is currently learning Rust and Substrate technologies.

Team Code Repositories

- **Web3 Labs:** https://github.com/web3labs
- **Web3j project:** https://github.com/web3j/

Future Plans

The present proposal exists to deliver a minimum viable product implementation of the Substrate Epirus Explorer and a public instance connected to a development network.

After this delivery, we expect to continue evolving the project and use each increment as feedback to the next phase to adapt the course of action as the problem space unfolds.

Down below is an illustrative sketch of the phases that could follow.

Phase 2 - Marketable System

- **Estimated duration:** 2 months
- **FTE:** 2
- **Costs:** 75,000 EUR

Tentative scope:

- Refinement of core abstractions and modules
- Next increment of the Contract module
- Ability to display different contract widgets based on contract metadata or standard implementations; e.g. PSP22 and PSP34
- Support for branded themes

Phase 3 - Web-Scale System

- **Estimated duration:** 2 months
- **FTE:** 2
- **Costs:** 75,000 EUR

Tentative scope:

- Ready to support a large number of users and transaction volumes
- Performance analysis and tuning to cope with high volumes
- Potentially includes horizontal scalability capabilities, e.g. sharding, batch splitting, distributed processing, etc.
- Monitoring and scalable deployment infrastructure

On-Going Support

Post delivery it is anticipated that an ongoing maintenance and support grant will be required from the Polkadot Treasury in DOT to keep Epirus up to date with the ink! smart contract language and Substrate evolution in general.

Business and Community

Web3 Labs is passionate about supporting the wider Web3 community.

Web3j has been going since 2016, and we have offered a free version of our Epirus Explorer for EVM-based chains since 2019.

Our commercial model of the Epirus Explorer is a paid per volume and support software as a service. We plan to incorporate the Substrate Epirus Explorer into the offering.

Based on our experience with Web3j over the years, pure open-source is hard to sustain. We want to ensure that our Epirus service on Substrate chains will serve the community while being a self-sustained revenue-generating product.

The open-core of the Substrate Epirus Explorer, i.e. any grant funded development, will be available to the community without any financial constraints. Anyone will be able to run and extend its instances of the Explorer for its Substrate-based solutions.

Additional Information

In order to support ink! and Substrate, parts of Epirus Explorer could be open-sourced under an Apache 2.0 license following an “open core” model. Proprietary components, like EVM modules and chain intelligence, would be available exclusively via Web3 Labs Epirus offerings.

Details about the existing Epirus offering for EVM chains:

- Epirus Explorer Website: https://www.web3labs.com/epirus-explorer
- Palm Explorer: https://palm.epirus.io

Additionally, it is worth mentioning that the team already built a composable data browser prototype for Solid PODs and RDF sources in general.

- Code repository: https://gitlab.com/omd_public/assemblage
- Assemblage demo video: https://www.youtube.com/watch?v=hYFEBAdkzmc